Metody i algorytmy optymalizacji

dr inż. Mateusz Baran

Akademia Górniczo-Hutnicza

Lato 2026

Optymalizacja z ograniczeniami

Do tej pory rozważaliśmy problemy optymalizacji funkcji $f\colon \mathcal{A} \to \mathbb{R}$

$$\arg\min_{x \in \mathcal{X}} f(x)$$

gdzie $\mathcal{X} = \mathcal{A}$. Często mamy jednak do czynienia z sytuacjami, gdy $\mathcal{X}$ jest istotnym podzbiorem $\mathcal{A}$. Przykładowo przy dopasowaniu modeli statystycznych możemy chcieć, aby wektory określające kierunki były ortogonalne lub pewne macierze aby były dodatnio określone, bądź też rozważać wyłącznie modele niskiego rzędu. W przypadku optymalizacji dyskretnej możliwe jest, że nasza zmienna decyzyjna może przyjmować tylko wartości całkowite bądź nieujemne (np. ile załadujemy sztuk towaru).

Przykłady ograniczeń

Dwa najczęstsze typy ograniczeń to ograniczenia równościowe $(g(x) = 0)$ i nierównościowe $(h(x) \geq 0)$. Ograniczenia z ostrą nierównością $(h(x) > 0)$ są dość problematyczne i wymagają specjalizowanych metod.

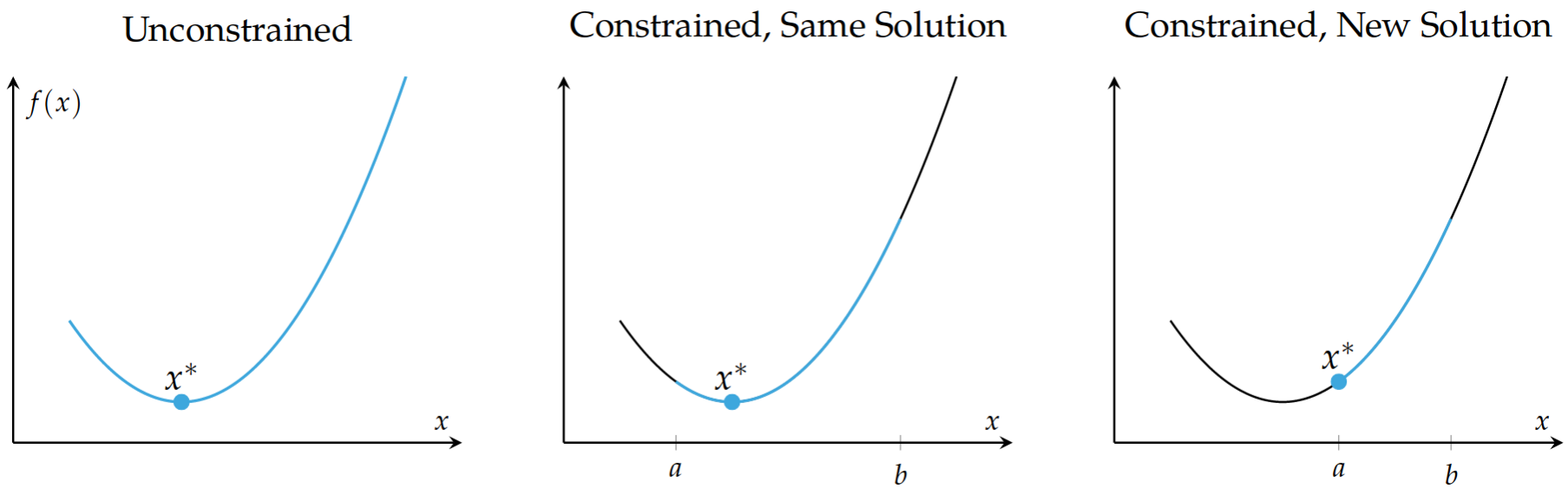

Figure: Źródło: Kochenderfer 2019

Jak radzimy sobie z ograniczeniami?

1. Wykonujemy reparametryzację, aby usunąć ograniczenia.
2. Wykorzystujemy specjalizowane algorytmy oparte np. o mnożniki Lagrange'a.
3. Relaksacja problemu.
4. Optymalizacja riemannowska (modyfikujemy kierunki przeszukiwania tak aby nie wyjść poza ograniczenia).
5. Szczególne przypadki, np. ograniczenia pudełkowe (*box constraints*).

Reparametryzacja

Na przykład ograniczenie

$$g(x) = 1 - \sum_{i=1}^{n} x_i$$

możemy zamienić na minimalizację

$$f_1([x_1, x_2, \ldots, x_{n-1}]) = f\left(\left[x_1, x_2, \ldots, x_{n-1}, 1 - \sum_{i=1}^{n-1} x_i\right]\right)$$

Mnożniki Lagrange'a

Rozważmy problem który ma tylko ograniczenia równościowe z ciągłymi pochodnymi. Możemy wtedy zastosować (podstawową wersję) metodę mnożników Lagrange'a.

$$\arg\min_{x\in\mathbb{R}^n} f(x)$$
$$g(x) = 0$$

Metoda opiera się o obserwację, że gradient f w optymalnym punkcie będzie miał ten sam kierunek co gradient g:

$$\nabla f(x) + \lambda \nabla g(x) = 0$$

lub, dla wielu ograniczeń,

$$\nabla f(x) + \sum_i \lambda_i \nabla g_i(x) = 0.$$

Pomocniczo wprowadzamy $\mathcal{L}(x,\lambda) = f(x) + \sum_i \lambda_i g_i(x)$, wtedy powyższy warunek sprowadza się do $\nabla\mathcal{L} = \nabla_x\mathcal{L} + \nabla_\lambda\mathcal{L} = 0$

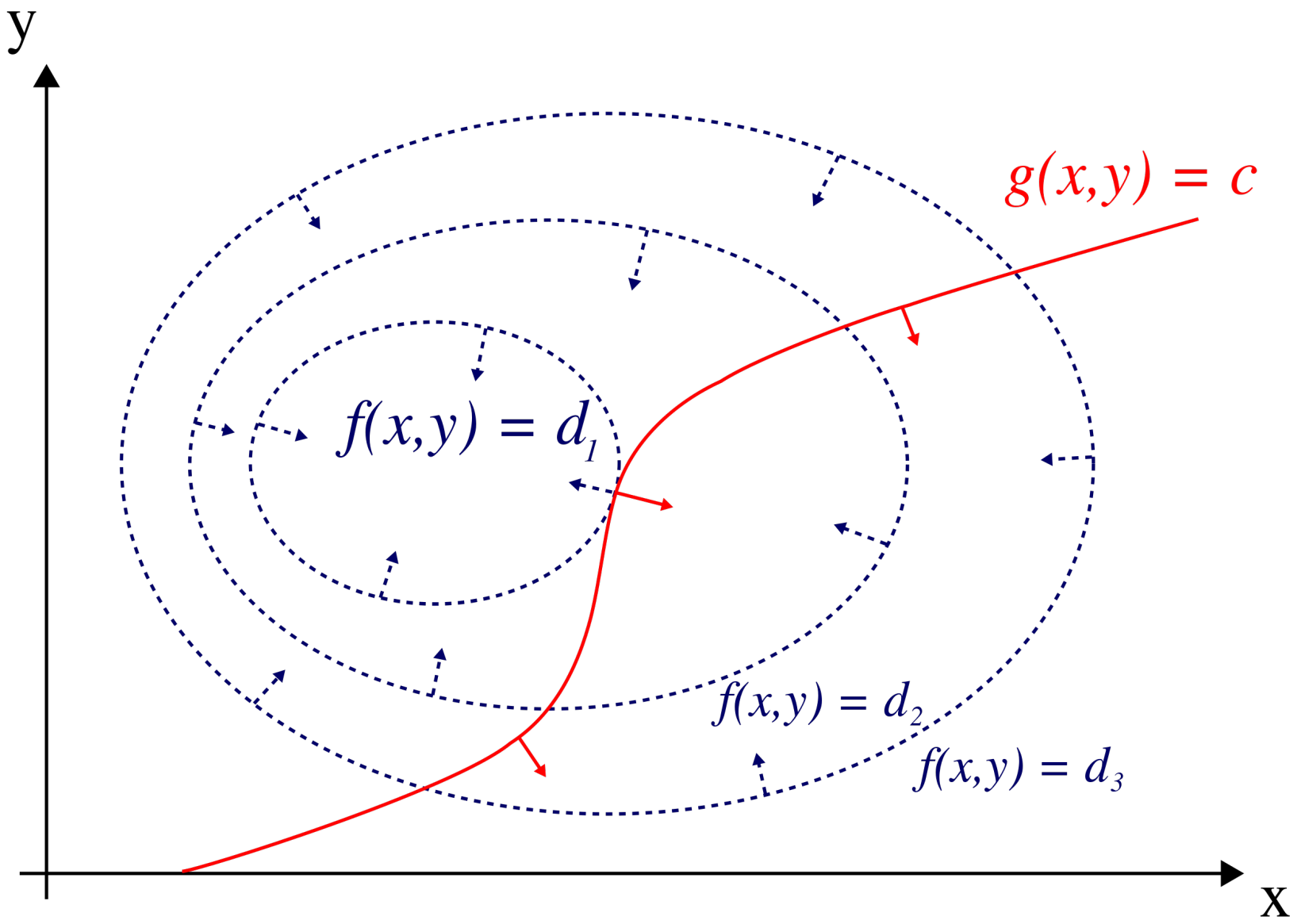

Figure: Źródło: `https://commons.wikimedia.org/wiki/File:LagrangeMultipliers2D.svg`

Ograniczenia nierównościowe

$$\arg\min_{x\in\mathbb{R}^n} f(x)$$
$$h(x) \leq 0$$

Rozróżniamy w tym momencie ograniczenia aktywne (gdy zachodzi równość $h(x) = 0$) i nieaktywne (ostra nierówność $h(x) < 0$).

Czasem zamienia się ograniczenia nierównościowe na równościowe przez wprowadzenie *slack variables* y_s (po jednej dla każdego ograniczenia):

$$h(x) + y_s = 0$$
$$y_s \geq 0$$

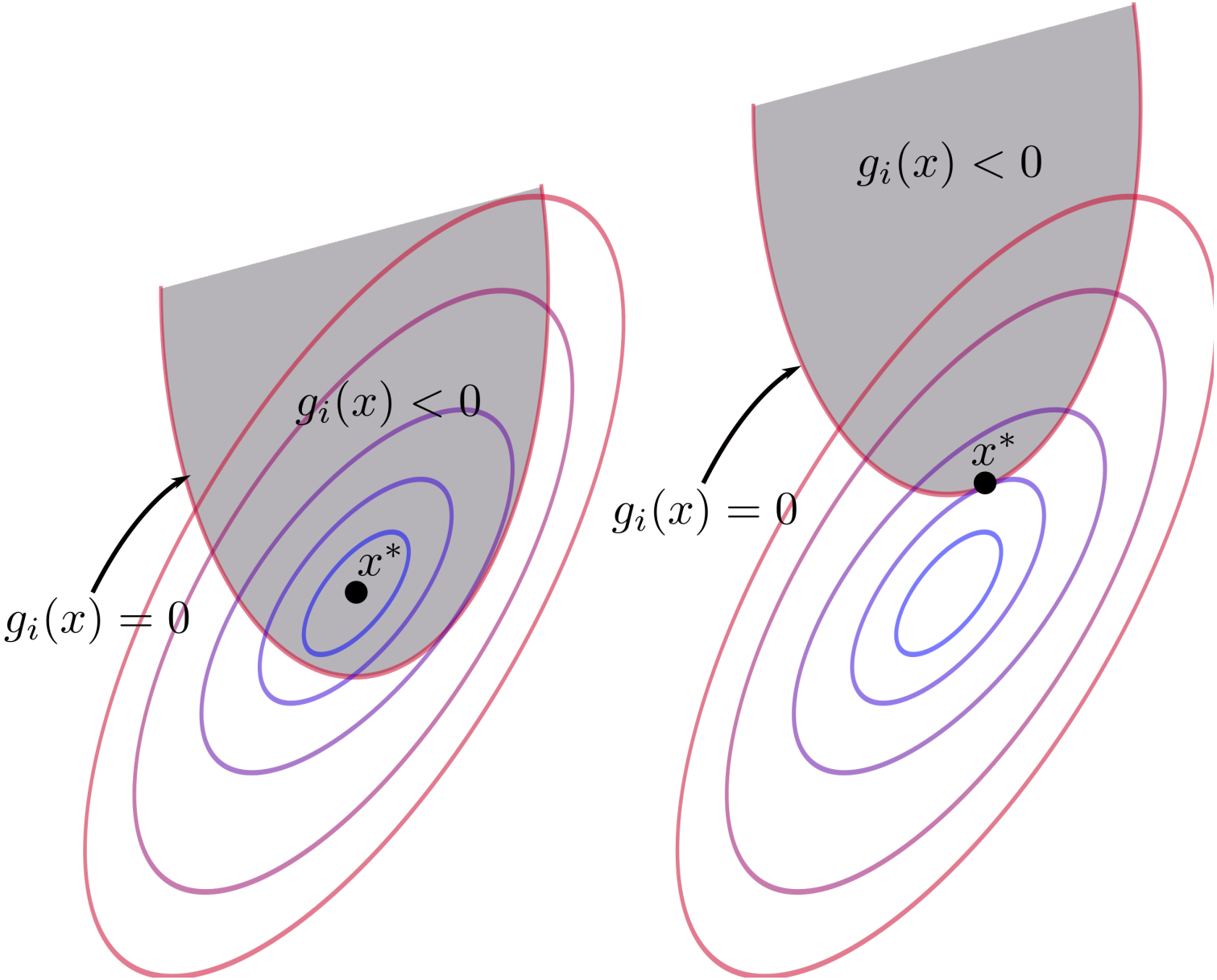

Figure: Źródło: `https://commons.wikimedia.org/wiki/File:Inequality_constraint_diagram.svg`

Moglibyśmy optymalizować $f_{\infty-\text{penalty}}(x) = f(x) + \infty(h(x) > 0)$, ale taka funkcja ma nieciągłość na brzegu dopuszczalnego obszaru i będąc w środku nie mamy informacji o tym, że się zbliżamy do tego brzegu.
Spróbujmy rozwiązać te dwa problemy wprowadzając skończony parametr kary $\mu \geq 0$ i dając nagrodę za oddalanie się od granicy:

$$\mathcal{L}(x, \mu) = f(x) + \mu h(x).$$

Zauważmy, że

$$f_{\infty-\text{penalty}}(x) = \max_{\mu \geq 0} \mathcal{L}(x, \mu),$$

bo w obszarze dopuszczalnym maksymalizujemy biorąc $\mu = 0$, a poza nim $\mathcal{L}$ może być dowolnie duże.
Możemy zatem sformułować problem

$$x^* = \arg\min_{x} \max_{\mu \geq 0} \mathcal{L}(x, \mu),$$

który nie jest prostszy od oryginalnego, ale pozwala nam na formułowanie koniecznych warunków optymalności (warunki Karusha–Kuhna–Tuckera) i projektowanie różnych metod rozwiązywania takich problemów.

Warunki (KKT) na punkt krytyczny x^*:

1. Dopuszczalność: $h(x^*) \leq 0$, $g(x^*) = 0$.
2. Kara jest nakładana za wychodzenie poza obszar dopuszczalny: $\mu \geq 0$.
3. *Complementary slackness*: dla każdego i zachodzi $\mu_i h_i(x^*) = 0$ (tylko aktywne ograniczenia mają niezerowy współczynnik μ_i).
4. Stacjonarność (gradient f jest komibnacją liniową gradientów aktywnych ograniczeń): $\nabla_x \mathcal{L} = 0$.

Metoda funkcji kary

Wymienione metody nie stanowią jednak gotowych algorytmów, a jedynie idee które nie są wykorzystywane przez wszystkie algorytmy, szczególnie w prostszych metodach jak np. metoda funkcji kary.
Możemy zamienić optymalizację z ograniczeniami

$$\begin{aligned} \arg\min_{x\in\mathbb{R}^n} f(x) \\ g(x) = 0 \\ h(x) \leq 0 \end{aligned}$$

na optymlizację bez ograniczeń

$$\begin{aligned} \arg\min_{x\in\mathbb{R}^n} f_p(x) \\ f_p(x) = f(x) + \rho p(x), \end{aligned}$$

gdzie p jest funkcją kary a $\rho > 0$ określa jej wielkość. Dla dostatecznie dużej wartości ρ rozwiązanie tego problemu będzie dobrze przybliżało rozwiązanie problemu z ograniczeniami.

Przykładowe funkcje kary:

$$p_{\text{count}}(x) = \sum_i (g_i(x) \neq 0) + \sum_j (h_j(x) > 0)$$

$$p_{\text{quadratic}}(x) = \sum_i (g_i(x))^2 + \sum_j \max(h_j(x), 0)^2$$

Przykładowa implementacja

```
function penalty_method(f, p, x0, k_max; ρ=1.0, γ=2.0,
    atol=eps(eltype(x0))
)
    x = x0
    for k in 1 : k_max
        x = minimize(x -> f(x) + ρ*p(x), x)
        ρ *= γ
        if p(x) < atol
            return x
        end
    end
    return x
end
```

`minimize` jest funkcją realizującą optymalizację funkcji jednej zmiennej. Należy zauważyć, że typowo zwracane rozwiązania będą blisko zbioru rozwiązań dopuszczalnych, ale niekoniecznie będą się w nim zawierały.

Augmented Lagrange method

W tej metodzie (opiszemy wariant dla ograniczeń równościowych) stosujemy funkcję kary postaci

$$p_{\mathrm{ALM}}(x) = \frac{1}{2}\rho \sum_j (g_j(x))^2 - \sum_j \lambda_j^k g_j(x),$$

gdzie wektor liczb $\lambda^k = [\lambda_1^k, \ldots, \lambda_{N_e}^k]$ aproksymuje współczynniki Lagrange'a w k-tej iteracji algorytmu.
Policzmy gradient z funkcji Lagrange'a:

$$\begin{aligned}\nabla_x \mathcal{L} &= \nabla_x \left(f(x) + \frac{1}{2}\rho \sum_j (g_j(x))^2 - \sum_j \lambda_j^k g_j(x) \right) \\ &= \nabla_x f(x) + \sum_j (\rho g_j(x) - \lambda_j^k) \nabla_x g_j(x)\end{aligned}$$

Wektor λ jest początkowo zerowy.
Chcemy sprowadzić gradient z funkcji Lagrange'a do zera (warunek stacjonarności), więc modyfikujemy λ tak, aby zbliżyć się do właściwych wartości mnożników Lagrange'a:

$$\lambda^{k+1} = \lambda^k - \rho g(x),$$

gdzie k jest numerem iteracji.

```julia
function augmented_lagrange_method(f, g, x, k_max; ρ=1, γ=2)
    λ = zeros(length(g(x)))
    for k in 1:k_max
        # w ksiazce (Kochenderfer 2019) funkcja optymalizowana jest bledna
        fp(x) = f(x) + ρ/2*sum(g(x).^2) - dot(λ, g(x))
        x = minimize(fp, x)
        ρ .*= γ
        λ .-= ρ.*g(x)
    end
    return x
end
```

Metoda często potrafi zadziałać z mniejszymi wartościami ρ niż bazowa wersja algorytmu metody funkcji kary.

Metody punktu wewnętrznego

Odwrotnie do metod funkcji kary zachowują się metody punktu wewnętrznego: rozważane są wyłącznie dopuszczalne rozwiązania, jednak czasami możemy nigdy nie osiągnąć rozwiązania optymalnego.
Wzór na funkcję celu jest analogiczny:

$$\arg\min_{x\in\mathbb{R}^n} f_p(x)$$

$$f_p(x) = f(x) + \frac{1}{\rho} p_{\text{barrier}}(x),$$

gdzie $p_{\text{barrier}}(x)$ jest ciągłą, nieujemną funkcją zbliżającą się do nieskończoności w miarę jak x zbliża się do brzegu zbioru rozwiązań dopuszczalnych.
Przykładowo możemy mieć (dla ograniczeń nierównościowych)

$$p_{\text{barrier}}(x) = -\sum_i \frac{1}{h_i(x)}.$$

```julia
function interior_point_method(f, p, x; ρ=1, γ=2, ε=0.001)
    delta = Inf
    while delta > ε
        x′ = minimize(x -> f(x) + p(x)/ρ, x)
        delta = norm(x′ - x)
        x = x′
        ρ *= γ
    end
    return x
end
```

Przeszukiwanie w kierunku wykorzystywane w metodzie optymalizacji globalnej musi brać pod uwagę górne ograniczenie na krok określone przez odległość do brzegu zbioru rozwiązań dopuszczalnych.

Przykład: *Support Vector Machine*

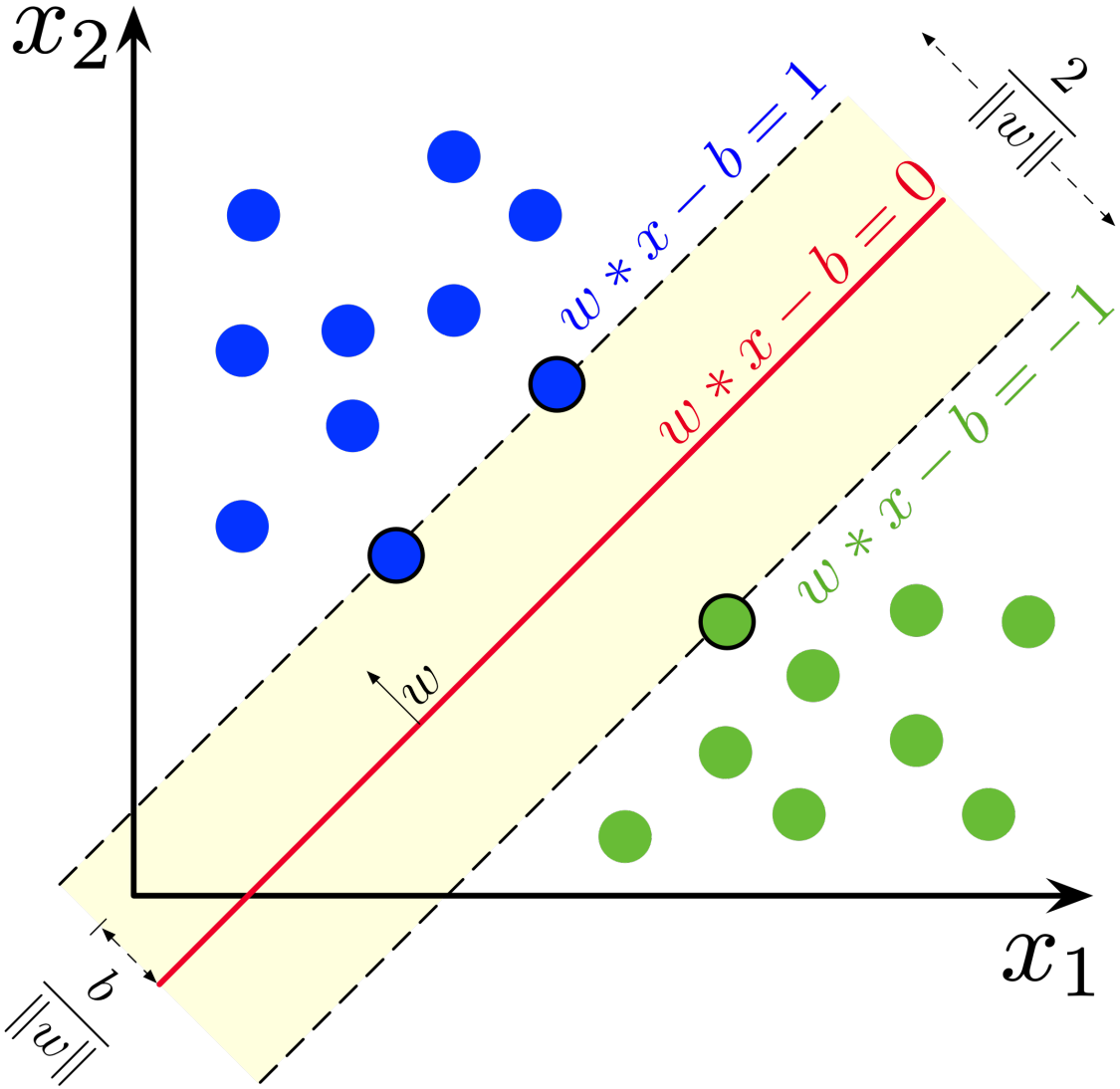

Figure: Źródło: `https://commons.wikimedia.org/wiki/File:SVM_margin.png`

Rozwiązywany problem (wariant z miękkim marginesem):

$$\begin{aligned} &\operatorname{argmin}_{\mathbf{w},b,\zeta} && \|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\zeta_i \\ &\text{przy ograniczeniach} && y_i(\mathbf{w}^\top\mathbf{x}_i - b) \geq 1 - \zeta_i, \quad \zeta_i \geq 0 \quad \forall i \in \{1,\ldots,n\} \end{aligned}$$

Gdzie: y_i to klasa i-tej próbki, $\mathbf{x}_i$ to wektor jej cech, a ζ_i określa o ile złamaliśmy przy niej margines.

Metody riemannowskie

Metody riemannowskie są klasą metod optymalizacji z ograniczeniami w której modyfikujemy operacje poruszania się po zbiorze dopuszczalnych rozwiązań tak, aby nigdy go nie opuścić. Ma to szereg zalet:

- Możemy rozważać ograniczenia nieciągłe (np. rząd macierzy).
- Możemy optymalizować cokolwiek, nie tylko wektory liczb.
- Problemy niewypukłe mogą stać się wypukłe.

oraz pewne wady:

- Nie wszystkie ograniczenia da się tak wyrazić i dalej musimy korzystać z wariantów omawianych wcześniej metod.
- Algorytmy riemannowskie są bardziej skomplikowane w implementacji i wyprowadzaniu. Na przykład riemannowska wersja metody punktu wewnętrznego została opublikowana w roku 2024 (Z. Lai and A. Yoshise, "Riemannian Interior Point Methods for Constrained Optimization on Manifolds." arXiv, Feb. 07, 2024. doi: 10.48550/arXiv.2203.09762).

Przykładowe problemy:

- Aproksymacja macierzy przez macierz nieujemną niskiego rzędu: znajdź macierze W, H minimalizujące $f(W, H) = \|V - WH\|^2$, gdzie W, H są macierzami nieujemnymi. Dodatkowo czasami zakłada się niski rząd macierzy W lub ortogonalność macierzy H ($HH^T = I$).
- Bayesowskie metody estymacji pozycji z ograniczeniami.

Algorytmy bezgradientowe do problemów z ograniczeniami: COBYLA, UOBYQA, NEWUOA, BOBYQA, LINCOA.

Tego typu algorytmy są skomplikowane w implementacji:

> *After almost three years of intensive coding, the modern Fortran version of PRIMA was finished by December 2022.*
> *In the past years, while working on PRIMA, I have spotted a dozen of bugs in reputable Fortran compilers and three bugs in MATLAB. Each of them represents days of bitter debugging, which finally led to the conclusion that it was not a problem in my code but a flaw in the Fortran compilers or in MATLAB. From a very unusual angle, this reflects how intensive the coding has been.*
> *The bitterness behind this "fun" fact is exactly why I work on PRIMA: I hope that all the frustrations that I have experienced will not happen to any user of Powell's methods anymore. I hope I am the last one in the world to decode a maze of 244 GOTOs in 7939 lines of Fortran 77 code — I did this for three years and I do not want anyone else to do it again.*

— `https://github.com/libprima/prima`